

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук
Кафедра программирования и информационных технологий

Разработка мобильного приложения для поиска рецептов коктейлей

Курсовая работа
09.03.02 Информационные системы и технологии
Профиль «Программная инженерия в информационных системах»

Зав. кафедрой	_____	д.ф.-м.н., доцент С. Д. Махортов
Обучающийся	_____	ст. 3 курса К. А. Ковалев
Руководитель	_____	асс. В. А. Ушаков

Воронеж 2026

СОДЕРЖАНИЕ

1 Предметная область и существующие решения.....	5
1.1 Характеристика предметной области.....	5
1.2 Цель.....	6
1.3 Обзор и сравнительный анализ существующих решений.....	7
1.3.1 Cocktail Flow.....	7
1.3.2 Drinkly.....	8
1.3.3 My Bar.....	9
1.3.4 Сравнительный анализ и выводы.....	10
2 Проектирование систем.....	12
2.1 Постановка задачи на разработку.....	12
2.1.1 Функциональные требования.....	12
2.1.2 Нефункциональные требования.....	14
2.1.3 Технологический стек и обоснование выбора.....	15
2.2 Архитектура системы.....	18
2.3 Проектирование базы данных.....	19
2.4 Проектирование модуля подбора рецептов по ингредиентам.....	20
2.5 Вывод по проектной части.....	21
3 Реализация разрабатываемой системы.....	23
3.1 Реализация пользовательского интерфейса.....	23
3.2 Реализация интеграции с API и хранение данных.....	25
3.3 Реализация модуля подбора рецептов по ингредиентам.....	27
3.4 Выводы по реализационной части.....	27
ЗАКЛЮЧЕНИЕ.....	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	31

ВВЕДЕНИЕ

В современном мире мобильные приложения стали неотъемлемой частью повседневной жизни человека. Они активно применяются не только для общения и развлечений, но и для решения сугубо практических бытовых задач. Одной из таких задач является поиск и подбор рецептов — в том числе рецептов коктейлей, интерес к домашнему приготовлению которых в последние годы заметно вырос.

Традиционный подход к поиску рецептов предполагает ручной просмотр множества сайтов и самостоятельное сопоставление состава рецептов с имеющимися ингредиентами. Такой процесс требует значительных временных затрат и не гарантирует результата.

Появление открытых баз рецептов с программным интерфейсом доступа кардинально изменило сложившуюся ситуацию. Стало возможным создавать мобильные приложения, которые не хранят рецепты самостоятельно, а получают их по запросу из внешнего источника — что обеспечивает актуальность данных и существенно снижает требования к ресурсам устройства. Тем не менее, даже при наличии подходящего источника данных, задача подбора рецептов сразу по нескольким ингредиентам остается технически нетривиальной и требует отдельного алгоритмического решения на стороне клиента.

Данное ограничение определяет ключевую техническую задачу разработки — реализацию клиентского алгоритма параллельного выполнения запросов с последующим объединением и дедупликацией результатов. Такой подход позволяет пользователю указать весь перечень имеющихся ингредиентов и получить полную подборку доступных рецептов без необходимости искать каждый ингредиент вручную. Пользовательские данные — избранные рецепты и список бара — хранятся локально без обязательной регистрации, а при авторизации через Google синхронизируются с облачным хранилищем Firebase Firestore, что обеспечивает доступ к ним с любого

устройства.

Однако большинство существующих мобильных приложений для работы с рецептами коктейлей либо предоставляют функцию подбора по ингредиентам только в платной версии, либо не реализуют её вовсе. Пользователи вынуждены выбирать между функциональными, но платными решениями и бесплатными каталогами без персонализации. Это создает потребность в разработке доступного приложения, которое сочетало бы в себе обширную базу рецептов, подбор по ингредиентам и синхронизацию данных без обязательной оплаты и регистрации.

Таким образом, разработка мобильного приложения для поиска рецептов коктейлей с функцией подбора по имеющимся ингредиентам является актуальной задачей, имеющей практическую значимость для широкой аудитории пользователей.

1 Предметная область и существующие решения

1.1 Характеристика предметной области

Предметная область данной работы — поиск, фильтрация и хранение рецептов коктейлей в мобильном приложении. Рецепт коктейля представляет собой структурированный набор данных: название, категория, признак алкогольности, перечень ингредиентов с указанием мер и инструкция по приготовлению. В отличие от классической кулинарии, коктейльные рецепты компактны по составу, однако предъявляют особые требования к точности пропорций и последовательности приготовления.

Ключевой особенностью предметной области является то, что рецепты не создаются самим приложением, а поступают из внешнего источника. Это накладывает ряд ограничений на архитектуру: структура данных фиксирована на стороне источника, возможности серверной фильтрации могут быть ограничены, а названия ингредиентов нередко содержат торговые марки вместо общих наименований. Все эти особенности напрямую влияют на проектные решения, принятые в ходе разработки.

Помимо работы с рецептами, предметная область включает управление персональными данными пользователя: списком избранных рецептов и перечнем ингредиентов домашнего бара. Эти данные хранятся локально на устройстве и не требуют регистрации, что снижает порог входа и делает приложение доступным без лишних шагов. При наличии авторизации данные синхронизируются с облачным хранилищем, обеспечивая доступ с нескольких устройств.

На рисунке 1 представлена схема основных процессов приложения.

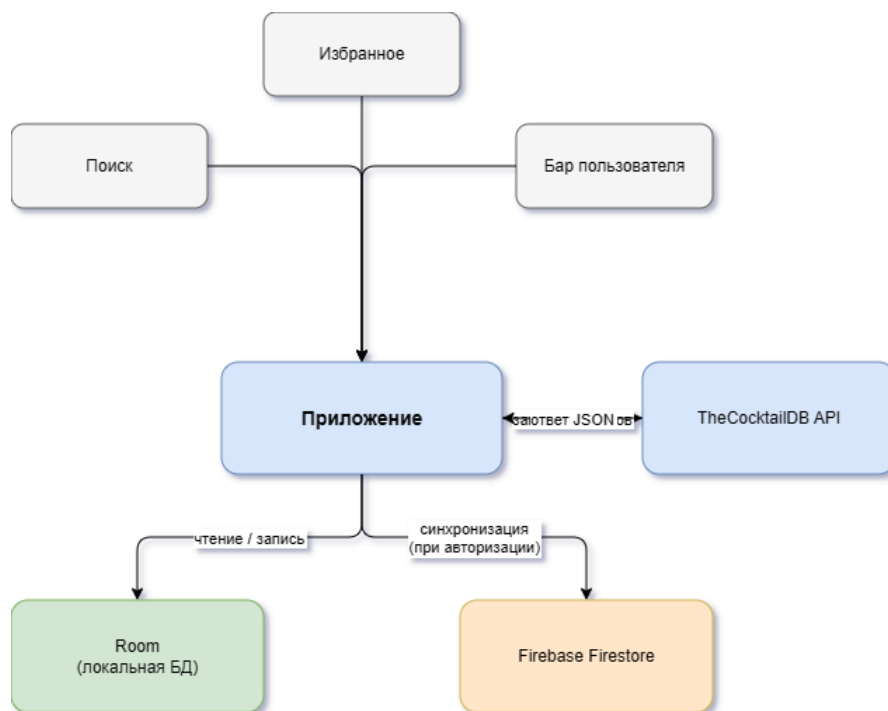


Рисунок 1 — Схема основных процессов приложения

1.2 Цель

Цель работы — разработка мобильного приложения для поиска рецептов коктейлей с функцией подбора по ингредиентам, имеющимся у пользователя дома.

Для достижения поставленной цели были решены следующие задачи:

1. Провести анализ предметной области и изучить существующие аналоги.
2. Сформулировать функциональные и нефункциональные требования к приложению.
3. Обосновать выбор технологического стека.
4. Спроектировать архитектуру приложения и структуру хранения данных.
5. Разработать алгоритм подбора рецептов по ингредиентам бара пользователя.
6. Реализовать пять основных экранов приложения.

7. Интегрировать внешний источник данных о рецептах.
8. Реализовать локальное хранение данных и облачную синхронизацию.

Объект исследования — процессы поиска, фильтрации и хранения рецептов коктейлей в мобильных приложениях.

Предмет исследования — методы и средства разработки мобильных приложений с интеграцией внешних источников данных, реализацией клиентских алгоритмов агрегации и локальным хранением пользовательского контекста.

1.3 Обзор и сравнительный анализ существующих решений

На современном рынке мобильных приложений для поиска рецептов коктейлей представлено несколько решений — от простых каталогов до приложений с функцией подбора по ингредиентам. В данном разделе проводится анализ наиболее популярных аналогов, выявляются их преимущества и недостатки, а также определяется место разрабатываемого приложения среди существующих продуктов. Анализ проводился на основе описаний приложений в Google Play и ручного тестирования основных пользовательских сценариев.

1.3.1 Cocktail Flow

Cocktail Flow — одно из наиболее известных приложений в категории, доступное на нескольких платформах. Приложение предлагает обширную базу рецептов с качественным визуальным оформлением и поддерживает поиск по названию и фильтрацию по категориям.

Основные функции:

— просмотр рецептов с фотографиями и пошаговыми инструкциями;

- фильтрация по категориям и крепости напитка;
- сохранение рецептов в избранное;
- функция подбора рецептов по имеющимся ингредиентам.

Преимущества:

- качественный дизайн
- удобная навигация
- большая база рецептов.

Недостатки:

- функция подбора по ингредиентам доступна только в платной версии

Вывод: функциональное, но частично платное решение. Ключевая функция подбора по ингредиентам недоступна без оплаты, что существенно ограничивает его применимость для широкой аудитории [3].

1.3.2 Drinkly

Drinkly — более простое приложение, ориентированное на casual-пользователя. Предлагает базу рецептов с возможностью поиска и сохранения избранного.

Основные возможности:

- поиск коктейлей по названию;
- сохранение рецептов в избранное;
- простой и понятный интерфейс.

Преимущества:

- бесплатное использование

- низкий порог входа
- лаконичный интерфейс

Недостатки:

- ограниченная база рецептов
- отсутствие функции подбора по нескольким ингредиентам
- нет синхронизации данных между устройствами

Вывод: доступное решение для базового просмотра рецептов, однако отсутствие персонализации и устаревшая база рецептов делают его недостаточным для полноценной работы [4].

1.3.3 My Bar

My Bar — приложение, специализирующееся на функции домашнего бара. Пользователь указывает доступные ингредиенты, приложение формирует список того, что можно приготовить.

Основные возможности:

- ведение списка ингредиентов домашнего бара;
- подбор рецептов по имеющимся ингредиентам;
- просмотр полных рецептов с инструкциями

Преимущества:

- функция бара реализована как основная, а не второстепенная;
- удобно для регулярного использования.

Недостатки:

- база рецептов значительно меньше, чем у конкурентов;
- интерфейс устарел;

— отсутствует синхронизация с облаком;

Вывод: узкоспециализированное, но устаревшее решение. Небольшая база рецептов и отсутствие синхронизации снижают его практическую ценность для современного пользователя [5].

1.3.4 Сравнительный анализ и выводы

Для наглядного сравнения рассмотренных решений с разрабатываемым приложением составлена таблица 1

Таблица 1 — Сравнительный анализ приложений для поиска рецептов коктейлей

Критерий	Cocktail Flow	Drinkly	My Bar	Разрабатываемое приложение
Бесплатное использование	Частично платно	Бесплатно	Бесплатно	Бесплатно
Большая база рецептов	Да	Нет	Нет	Да
Поиск по названию	Да	Да	Нет	Да
Подбор по ингредиентам	Платно	Нет	Да	Да
Облачная синхронизация	Нет	Нет	Нет	Да

Анализ таблицы показывает, что ни одно из рассмотренных решений не

сочетает одновременно большую базу рецептов, бесплатный подбор по ингредиентам и синхронизацию данных без обязательной регистрации. Разрабатываемое приложение отличается от аналогов именно этим сочетанием возможностей.

Результаты анализа подтверждают актуальность разработки и служат основанием для перехода к проектированию системы.

2 Проектирование систем

2.1 Постановка задачи на разработку

На основании результатов аналитической части и проведенного обзора существующих решений сформулирована задача на разработку мобильного приложения для поиска рецептов коктейлей.

Разработать мобильное приложение, предназначенное для поиска и хранения рецептов коктейлей с функцией подбора по ингредиентам, имеющимся у пользователя дома. Приложение должно обеспечивать удобную навигацию по обширной базе рецептов, персонализацию через избранное и бар пользователя, а также опциональную синхронизацию данных между устройствами без обязательной регистрации.

За рамками данной работы остались следующие функции:

- создание и редактирование пользовательских рецептов;
- социальные функции — комментарии, оценки, публикации;
- интеграция с магазинами для покупки ингредиентов;
- обязательная авторизация.

Их реализация может стать направлением дальнейшего развития продукта.

2.1.1 Функциональные требования

Функциональные требования определяют основные возможности системы и действия, которые пользователь может выполнять в процессе работы с приложением.

Каталог и просмотр рецептов. Приложение должно отображать коктейли в виде сетки с фотографиями и названиями. Список должен

поддерживать фильтрацию по категориям, загружаемым динамически из источника данных. При прокрутке до конца списка приложение должно автоматически отображать следующую порцию данных — подгрузка реализована на уровне клиента путём порционного отображения уже полученного списка. Пользователь должен иметь возможность открыть детальную карточку любого коктейля с полным составом и инструкцией по приготовлению, а также получить случайный рецепт по нажатию кнопки.

Поиск. Приложение должно поддерживать поиск рецептов по названию с отображением результатов в реальном времени. История последних поисковых запросов должна сохраняться и отображаться при пустом поисковом поле с возможностью её очистки одним действием. После получения результатов должна быть доступна фильтрация по признаку алкогольности — алкогольные, безалкогольные или все

Избранное. Приложение должно позволять добавлять коктейли в избранное и удалять их непосредственно с карточки. Список избранного должен отображаться в виде сетки аналогично каталогу. При отсутствии сохраненных рецептов приложение должно отображать пустое состояние с навигацией в каталог.

Бар пользователя. Приложение должно позволять пользователю формировать список ингредиентов, имеющихся дома. Добавление ингредиента должно осуществляться через диалог с поиском по полному списку доступных ингредиентов. При любом изменении списка приложение должно автоматически обновлять подборку доступных рецептов. Поиск должен выполняться параллельно по каждому ингредиенту с объединением результатов и удалением дубликатов.

Профиль и авторизация. Приложение должно работать без обязательной регистрации. Пользователь должен иметь возможность авторизоваться через Google для синхронизации данных между устройствами. На экране профиля должны отображаться счётчики избранных рецептов и ингредиентов бара, а также возможность очистить каждый из списков.

Синхронизация данных. При авторизации приложение должно синхронизировать избранное и список бара с облачным хранилищем. При выходе из аккаунта приложение должно продолжать работу с локальными данными без потери функциональности.

На рисунке 2 представлена диаграмма прецедентов, визуализирующая функциональные требования.

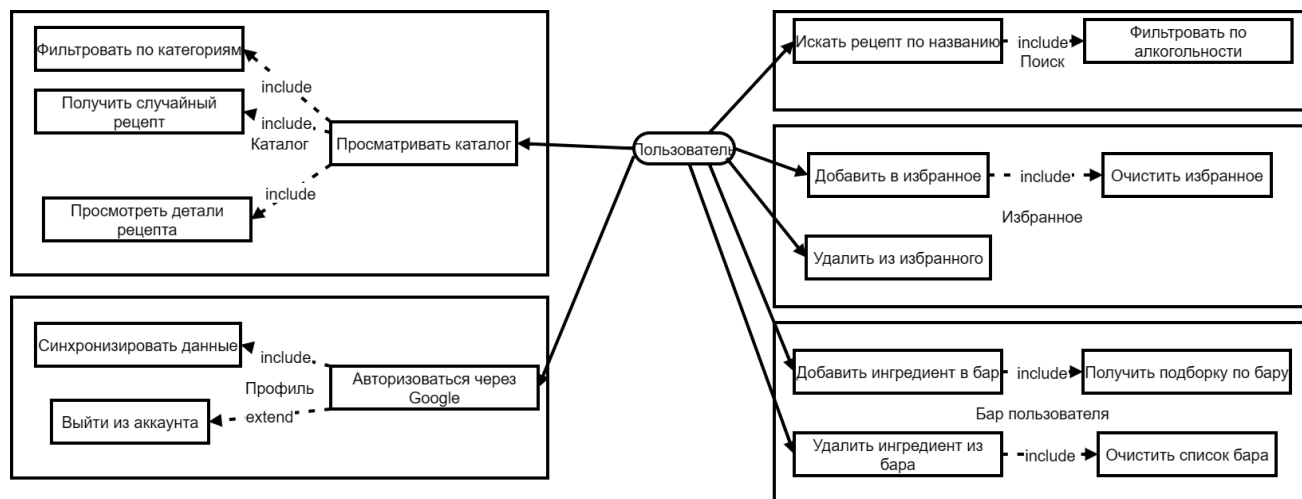


Рисунок 2 — Диаграмма прецедентов

2.1.2 Нефункциональные требования

Нефункциональные требования определяют качественные характеристики системы и условия её эксплуатации.

Требования к производительности. Приложение должно обеспечивать визуальную обратную связь при длительных операциях и выполнять сетевые запросы асинхронно, не блокируя интерфейс. Поиск по ингредиентам бара должен выполняться параллельно, чтобы суммарное время ожидания не зависело линейно от количества добавленных ингредиентов. Все операции с локальной базой данных должны выполняться в фоновом потоке.

Требования к надежности. Приложение не должно завершать работу с ошибкой при отсутствии интернет-соединения — вместо этого оно должно

отображать соответствующее сообщение и сохранять доступ к локально сохраненным данным. При временной недоступности внешнего источника данных приложение должно корректно обрабатывать ошибку и информировать пользователя. Локальные данные не должны теряться при обновлении приложения.

Требования к удобству использования. Навигация между экранами должна осуществляться через нижнюю панель вкладок, что соответствует стандартам Material Design и привычным паттернам поведения пользователей. Каждый экран должен иметь корректно оформленное пустое состояние с понятным текстом и подсказкой к действию. Интерфейс должен корректно отображаться на устройствах с различными размерами экрана.

Требования к совместимости. Приложение должно поддерживать актуальные версии мобильной операционной системы, охватывающие подавляющее большинство активных устройств. Сборка и развёртывание должны осуществляться стандартными средствами без дополнительных зависимостей от внешней инфраструктуры.

Требования к безопасности. Пользовательские данные в облачном хранилище должны быть доступны исключительно авторизованному владельцу. Локальная база данных не должна содержать чувствительных персональных данных.

2.1.3 Технологический стек и обоснование выбора

Для реализации приложения был выбран следующий технологический стек.

Язык программирования — Kotlin. Kotlin является официальным рекомендуемым языком разработки мобильных приложений на платформе Android по версии Google начиная с 2019 года [6]. По сравнению с Java он предлагает более лаконичный синтаксис, встроенную защиту от

NullPointerException и нативную поддержку корутин для асинхронного программирования.

Пользовательский интерфейс — Jetpack Compose. Jetpack Compose — современный декларативный фреймворк для построения UI, который Google активно развивает как основной инструмент для новых проектов [7]. В отличие от классического подхода с XML-разметкой, Compose позволяет описывать интерфейс непосредственно в коде, что упрощает работу с динамическими состояниями и сокращает количество boilerplate-кода. В качестве дизайн-системы использован Material Design 3.

Навигация — Navigation Compose. Библиотека обеспечивает единый граф навигации в рамках одной Activity, что соответствует архитектурным рекомендациям и упрощает управление back stack при переходах между экранами.

Внедрение зависимостей — Koin. По сравнению с Dagger/Hilt Koin проще в настройке и не требует кодогенерации, что ускоряет разработку. Все зависимости регистрируются в едином модуле и передаются в ViewModel через конструктор [8].

Сетевое взаимодействие — Retrofit, OkHttp и Gson. Retrofit является отраслевым стандартом для выполнения HTTP-запросов в мобильных приложениях. OkHttp выступает в роли HTTP-клиента, Gson используется для десериализации JSON-ответов в объекты Kotlin.

Локальная база данных — Room. Официальная библиотека-обёртка над SQLite от Google, обеспечивающая безопасную работу с локальной базой данных через аннотации и DAO-интерфейсы [9]. Используется для хранения избранных рецептов, списка ингредиентов бара и истории поисковых запросов.

Загрузка изображений — Coil. Библиотека разработана с нативной поддержкой Kotlin и Jetpack Compose. По сравнению с Glide и Picasso она лучше интегрируется с Compose-компонентами и имеет меньший размер.

Авторизация и облачное хранилище — Firebase. Firebase Authentication обеспечивает авторизацию через Google Sign-In. Firebase Firestore используется

для синхронизации избранного и списка бара между устройствами авторизованного пользователя.

В таблице 2 представлен сводный обзор технологического стека.

Таблица 2 — Технологический стек приложения

Компонент	Технология	Обоснование
Язык	Kotlin	Официальный язык платформы
Пользовательский интерфейс	Jetpack Compose, Material 3	Декларативный подход, актуальный стандарт
Навигация	Navigation Compose	Единый граф навигации
Внедрение зависимостей	Koin	Простота настройки
Сеть	Retrofit, OkHttp, Gson	Отраслевой стандарт для работы с REST API
Локальная база данных	Room	Официальная библиотека
Изображения	Coil	Нативная поддержка Compose, малый вес
Авторизация	Firebase Auth	Вход с использованием аккаунта Google
Облако	Firebase Firestore	Синхронизация данных в реальном времени
Асинхронность	Coroutines	Нативный инструмент для фоновых задач

2.2 Архитектура системы

Приложение построено на основе Clean Architecture — подхода, при котором код разделяется на независимые слои с чётко разграниченной ответственностью. Основная идея состоит в том, что внутренние слои не зависят от внешних: бизнес-логика не знает ни о фреймворках, ни о базе данных, ни о пользовательском интерфейсе [10]. Такой подход обеспечивает независимое тестирование каждого слоя и упрощает замену отдельных компонентов без изменения остальной части системы.

Архитектура включает три основных слоя.

Слой данных (data) отвечает за получение и хранение данных из всех источников: удалённого API через Retrofit, локальной базы данных через Room и облачного хранилища через Firebase Firestore. В этом слое реализованы маппер-функции, преобразующие DTO-модели в доменные объекты, а репозитории реализуют интерфейсы, определённые в доменном слое.

Доменный слой (domain) является центральным и не зависит ни от платформы, ни от конкретных библиотек. Он содержит доменные модели — Cocktail, CocktailPreview, Ingredient — и интерфейсы репозиториев. Бизнес-логика инкапсулирована в use case-классах: каждый из них отвечает за одно конкретное действие, например получение случайного коктейля или переключение состояния избранного.

Слой представления (presentation) реализован по паттерну MVVM: каждый экран имеет соответствующий ViewModel, который управляет состоянием UI и взаимодействует с доменным слоем исключительно через use case-ы. View-компоненты на Jetpack Compose подписываются на состояние ViewModel и перерисовываются при его изменении.

На рисунке 3 представлена архитектурная схема приложения.

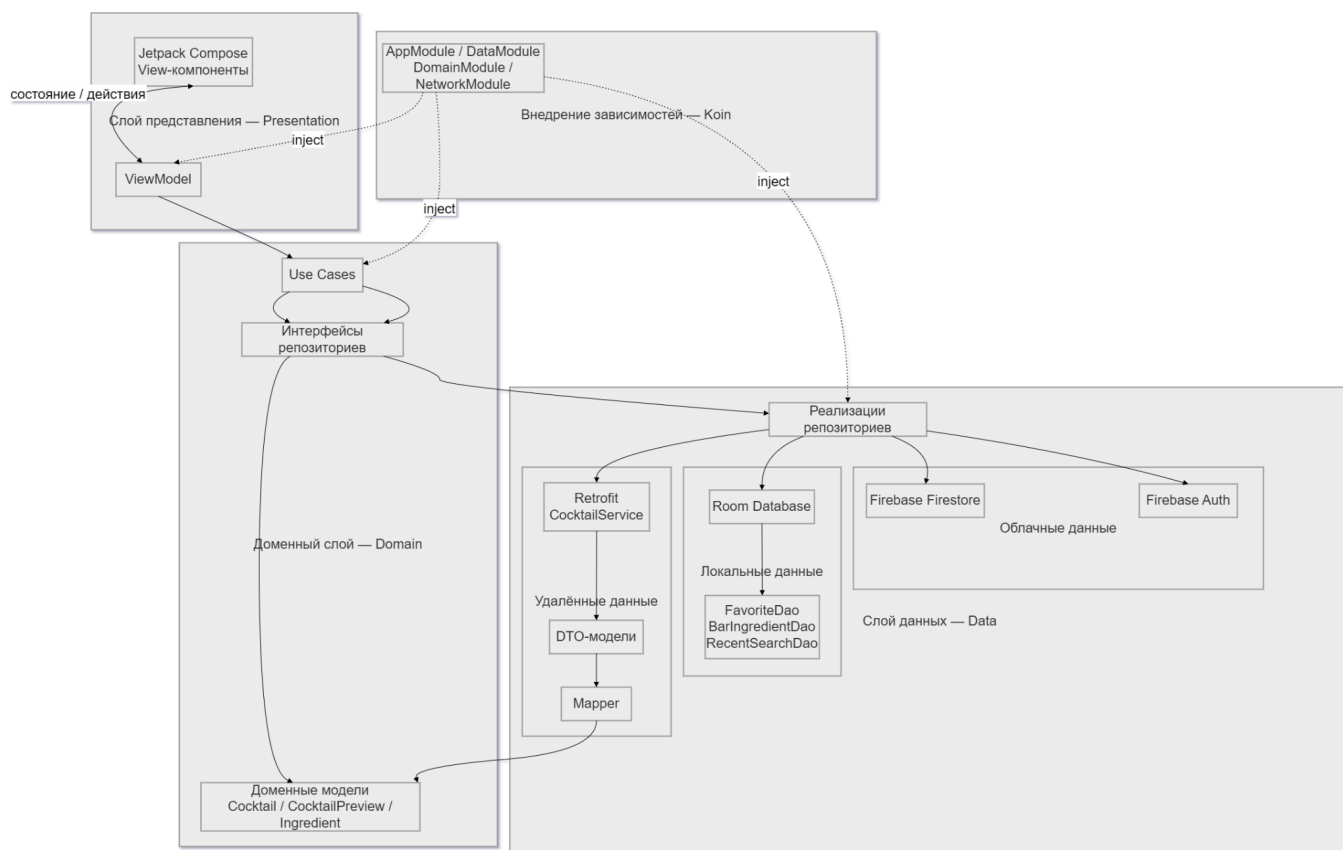


Рисунок 3 — Архитектурная схема приложения

Структура пакетов организована по фичам, а не по типам классов. Каждая функциональная часть — каталог, поиск, избранное, бар, профиль, детали — содержит собственные `View` и `ViewModel`. Общие компоненты, тема и навигация вынесены в отдельные пакеты. Все зависимости регистрируются в `Koin`-модулях и передаются через конструкторы, что исключает жёсткую связанность и упрощает замену реализаций.

На рисунке 4 представлена диаграмма классов приложения.

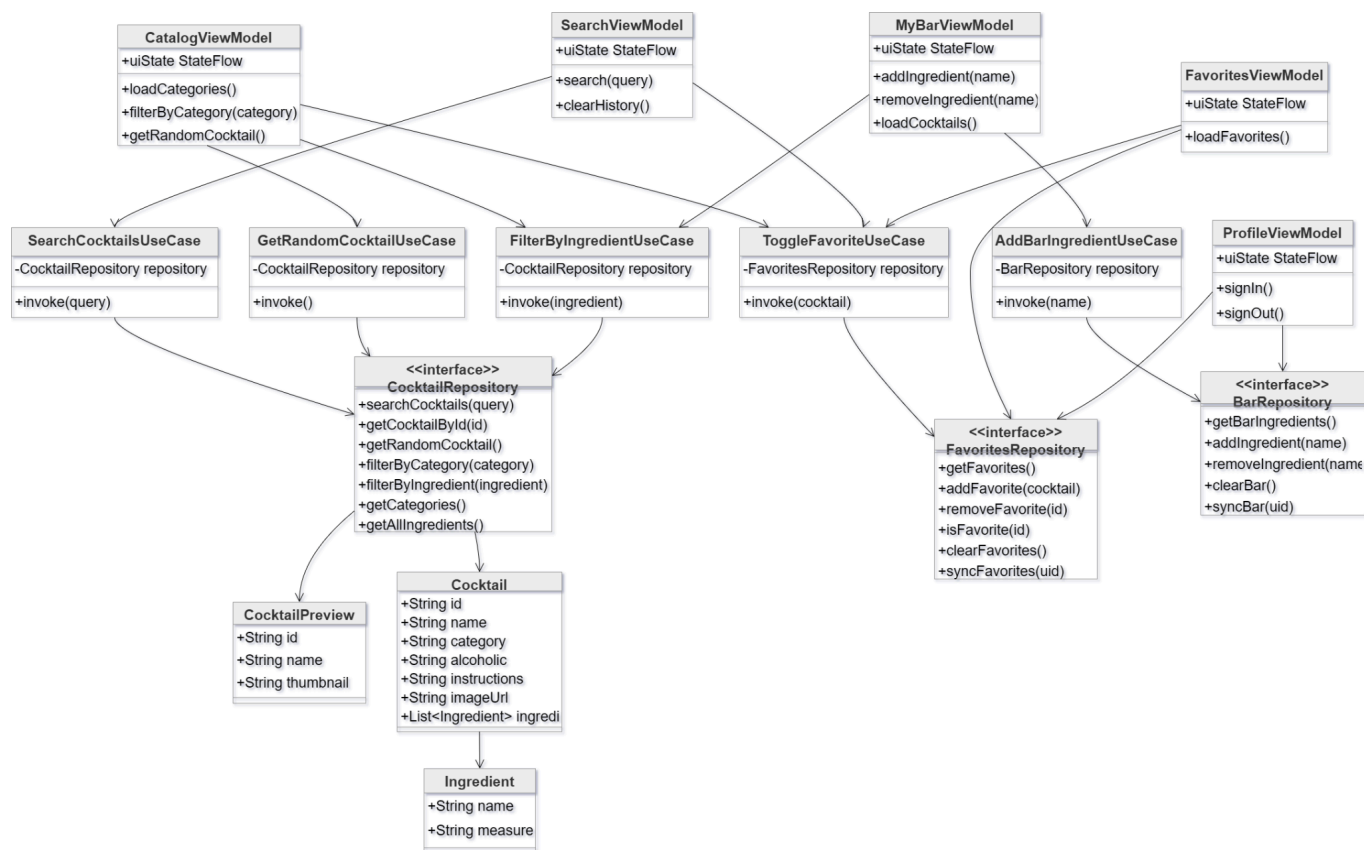


Рисунок 4 — Диаграмма классов приложения

2.3 Проектирование базы данных

Приложение использует два независимых хранилища данных: локальную базу на устройстве через Room и облачное хранилище Firebase Firestore. Выбор активного хранилища определяется состоянием авторизации и происходит прозрачно для остальных компонентов системы.

Локальная база данных содержит три таблицы. Таблица `favorites` хранит избранные коктейли пользователя — идентификатор, название и ссылку на миниатюру. Полная информация о коктейле всегда может быть получена из внешнего источника по идентификатору, поэтому хранить её локально избыточно. Таблица `bar_ingredients` хранит названия ингредиентов, добавленных в бар, — этого достаточно для формирования поисковых запросов. Таблица `recent_searches` хранит историю поисковых запросов с временными

метками для отображения в хронологическом порядке.

На рисунке 5 представлена ER-диаграмма базы данных.

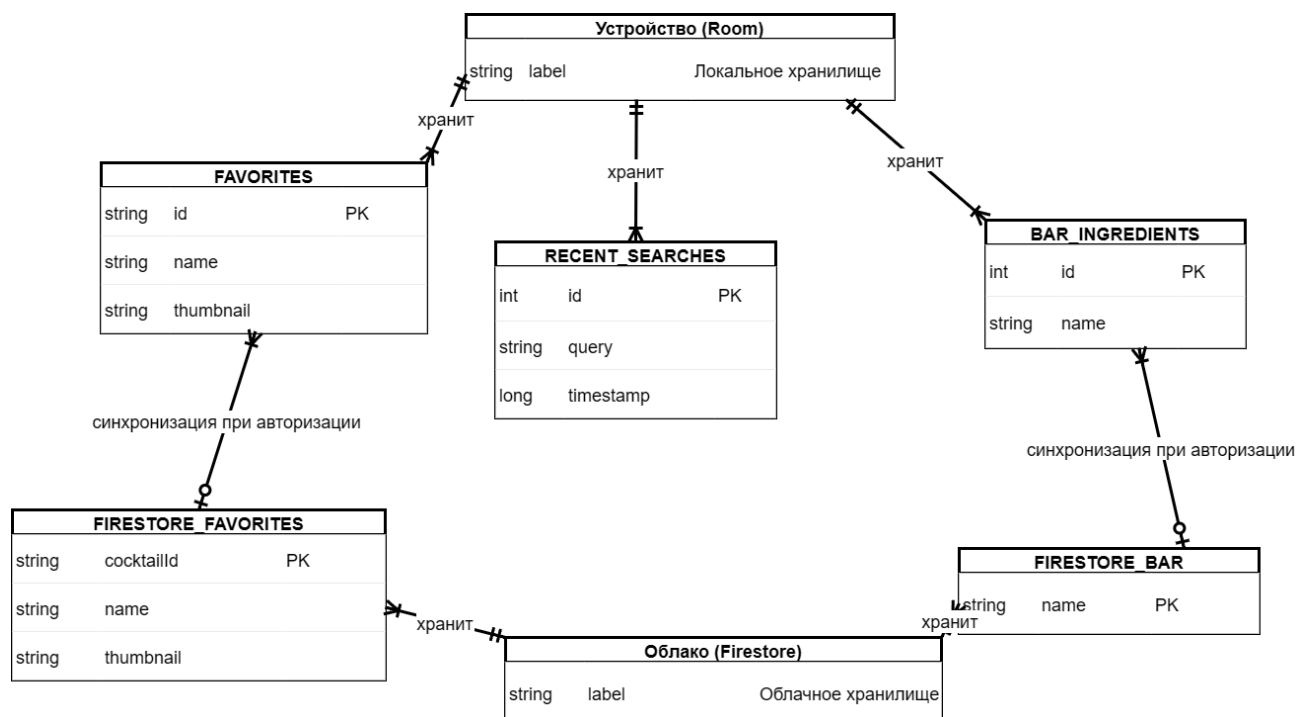


Рисунок 5 — ER-диаграмма базы данных

Облачное хранилище используется исключительно при авторизации пользователя и служит для синхронизации данных между устройствами. Структура коллекций повторяет структуру локальных таблиц: коллекция favorites хранит избранные коктейли, коллекция bar — список ингредиентов. Доступ к данным разграничен на уровне правил безопасности: пользователь может читать и записывать только свои данные.

При авторизации выполняется объединение локальных и облачных данных с удалением дубликатов. При выходе из аккаунта приложение продолжает работу с локальными данными без потери функциональности.

2.4 Проектирование модуля подбора рецептов по ингредиентам

Модуль подбора рецептов по ингредиентам является ключевой функциональной особенностью приложения. Его задача — сформировать список коктейлей, которые пользователь может приготовить из ингредиентов, имеющихся в его баре.

Внешний источник данных допускает фильтрацию только по одному ингредиенту за запрос. Для решения этой задачи был спроектирован алгоритм параллельного выполнения запросов: для каждого ингредиента из бара формируется отдельный запрос, все запросы выполняются одновременно через механизм корутин, после чего результаты объединяются и дубликаты удаляются по идентификатору коктейля. Параллельное выполнение снижает суммарное время ожидания по сравнению с последовательным подходом.

Для предотвращения избыточных запросов при быстром добавлении нескольких ингредиентов подряд в алгоритм введена задержка — `debounce 400` мс. Запрос отправляется только после того, как пользователь прекратил вносить изменения в список бара.

На рисунке 6 представлена схема работы алгоритма подбора рецептов.

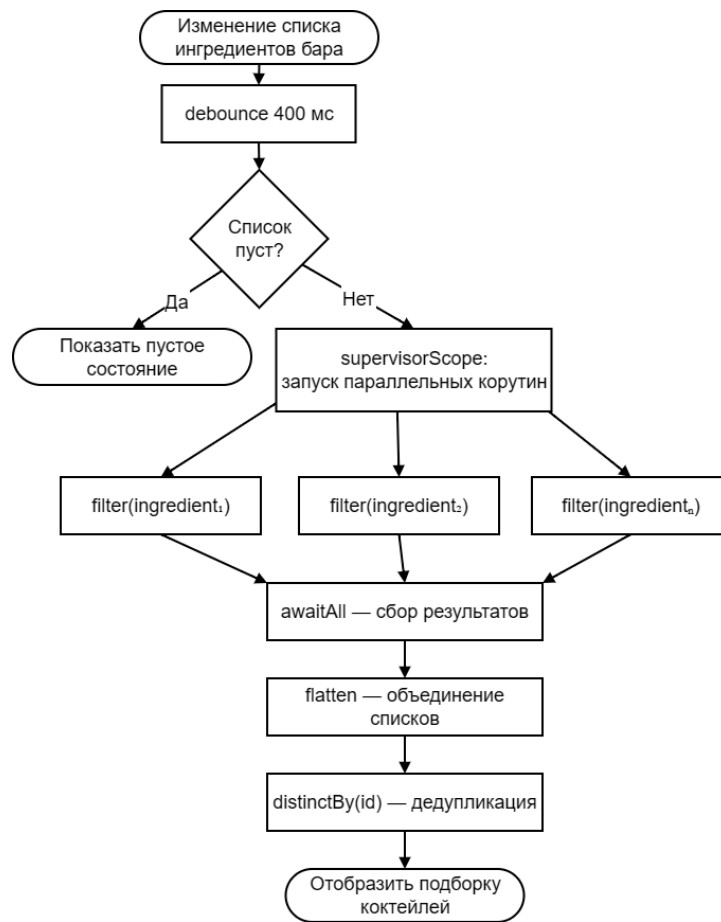


Рисунок 6 — Схема работы алгоритма подбора рецептов

2.5 Вывод по проектной части

В результате выполнения проектной части работы были решены следующие задачи:

- сформулированы функциональные и нефункциональные требования, охватывающие все основные сценарии использования приложения;
- обоснован технологический стек: Kotlin, Jetpack Compose, Retrofit, Room, Koin, Firebase;
- спроектирована архитектура на основе Clean Architecture с разделением на слои данных, домена и представления;

- спроектирована структура локальной базы данных и определена схема облачного хранилища;
- спроектирован алгоритм параллельного подбора рецептов по ингредиентам с объединением результатов и устранением дубликатов.

Принятые решения обеспечивают слабую связанность компонентов, возможность независимого тестирования бизнес-логики и простоту расширения функциональности. Полученные результаты служат основанием для перехода к программной реализации.

3 Реализация разрабатываемой системы

3.1 Реализация пользовательского интерфейса

Пользовательский интерфейс приложения реализован на Jetpack Compose с использованием Material Design 3. Навигация между основными разделами осуществляется через нижнюю панель вкладок, содержащую пять пунктов: каталог, поиск, избранное, мой бар и профиль. Такой подход соответствует стандартным паттернам навигации в мобильных приложениях и обеспечивает быстрый доступ к любому разделу из любой точки приложения.

Экран каталога является стартовым экраном приложения. Коктейли отображаются в виде сетки из двух колонок — каждая карточка содержит фотографию, название и индикатор избранного, который обновляется без перезагрузки экрана. В верхней части расположена горизонтальная лента чипов для фильтрации по категориям, загружаемым динамически. При прокрутке до конца списка автоматически подгружается следующая порция данных.

На рисунке 7 представлен экран каталога.

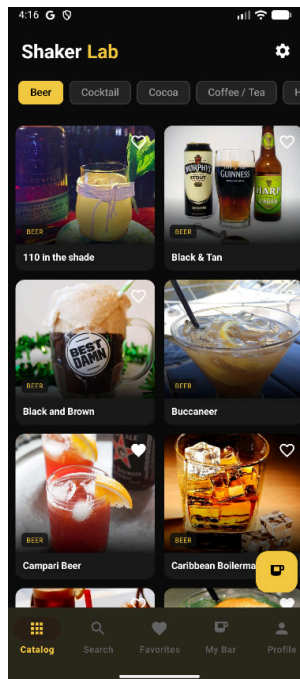


Рисунок 7 — Экран каталога

Экран поиска содержит текстовое поле, при пустом значении которого отображается история последних запросов в виде горизонтальной ленты чипов. После ввода запроса и получения результатов появляются чипы фильтрации по признаку алкогольности. Фильтрация выполняется на стороне клиента без дополнительных сетевых запросов.

На рисунке 8 представлен экран поиска.

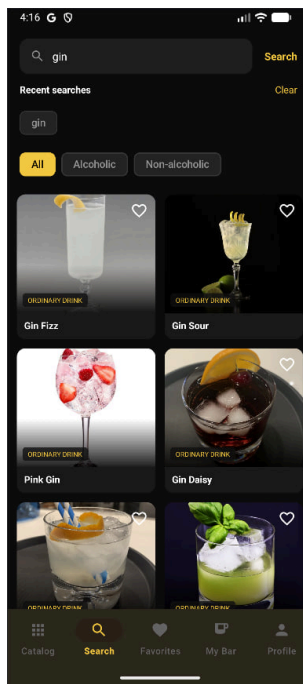


Рисунок 8 — Экран поиска

Экран избранного по структуре аналогичен каталогу — та же сетка карточек с возможностью удаления рецепта из избранного прямо отсюда. При отсутствии сохранённых рецептов отображается пустое состояние с кнопкой перехода в каталог.

На рисунке 9 представлен экран избранного.

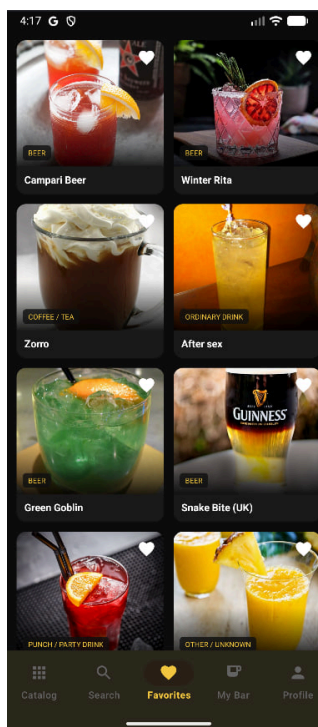


Рисунок 9 — Экран избранного

Экран «Мой бар» состоит из двух частей: горизонтальная лента чипов с добавленными ингредиентами и список подобранных коктейлей ниже. Добавление нового ингредиента осуществляется через диалог с поисковым полем и прокручиваемым списком всех доступных ингредиентов. При изменении списка бара подборка коктейлей обновляется автоматически. Экран поддерживает два пустых состояния: если список ингредиентов пуст — отображается приглашение добавить первый ингредиент; если ингредиенты добавлены, но подходящих рецептов не найдено — отображается соответствующее сообщение.

На рисунке 10 представлен экран мой бар.

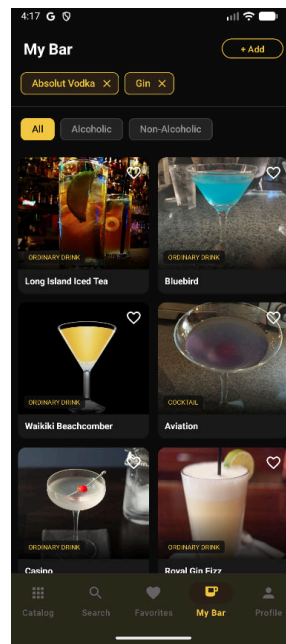


Рисунок 10 — Экран мой бар

Экран профиля отображает информацию об авторизованном пользователе или гостевой режим при отсутствии входа. Здесь же доступны счётчики избранных рецептов и ингредиентов бара, кнопки очистки каждого из списков и выход из аккаунта.

На рисунке 11 представлен экран профиля.

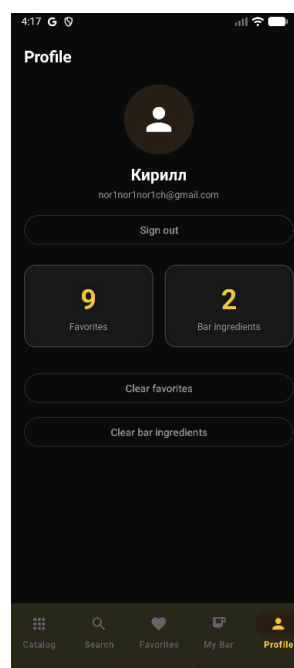


Рисунок 11 — Экран профиля

Детальный экран открывается при нажатии на любую карточку в приложении. Он содержит фотографию коктейля на всю ширину экрана, название, теги категории и алкогольности, список ингредиентов с мерами и инструкцию по приготовлению. Кнопка избранного и кнопка случайного коктейля расположены поверх контента в виде плавающих элементов.

На рисунке 12 представлен детальный экран коктейля.



Рисунок 12 — Детальный экран коктейля

3.2 Реализация интеграции с API и хранение данных

Взаимодействие с внешним источником данных реализовано через библиотеку Retrofit. Все запросы описаны в интерфейсе CocktailService, методы которого аннотированы соответствующими HTTP-методами и путями. Ответы десериализуются из JSON в DTO-модели через Gson, после чего маппер-функции преобразуют их в доменные модели. Такое разделение гарантирует, что изменения в структуре внешнего источника не затрагивают

доменный и презентационный слои.

В таблице 3 представлены используемые endpoints.

Таблица 3 — Используемые endpoints API

Endpoint	Назначение
search.php?s=	Поиск коктейлей по названию
lookup.php?i=	Получение коктейля по идентификатору
random.php	Получение случайного коктейля
filter.php?c=	Фильтрация по категории
filter.php?a=	Фильтрация по признаку алкогольности
filter.php?i=	Фильтрация по ингредиенту
list.php?c=list	Получение списка всех категорий
list.php?i=list	Получение списка всех ингредиентов

Все сетевые запросы выполняются асинхронно через корутины. Репозитории оборачивают вызовы к источнику данных в блоки обработки исключений, возвращая результат в виде sealed-класса с состояниями успеха и ошибки. Это позволяет ViewModel корректно обрабатывать сетевые ошибки и отображать пользователю понятные сообщения.

На рисунке 13 представлена схема взаимодействия компонентов при выполнении сетевого запроса.

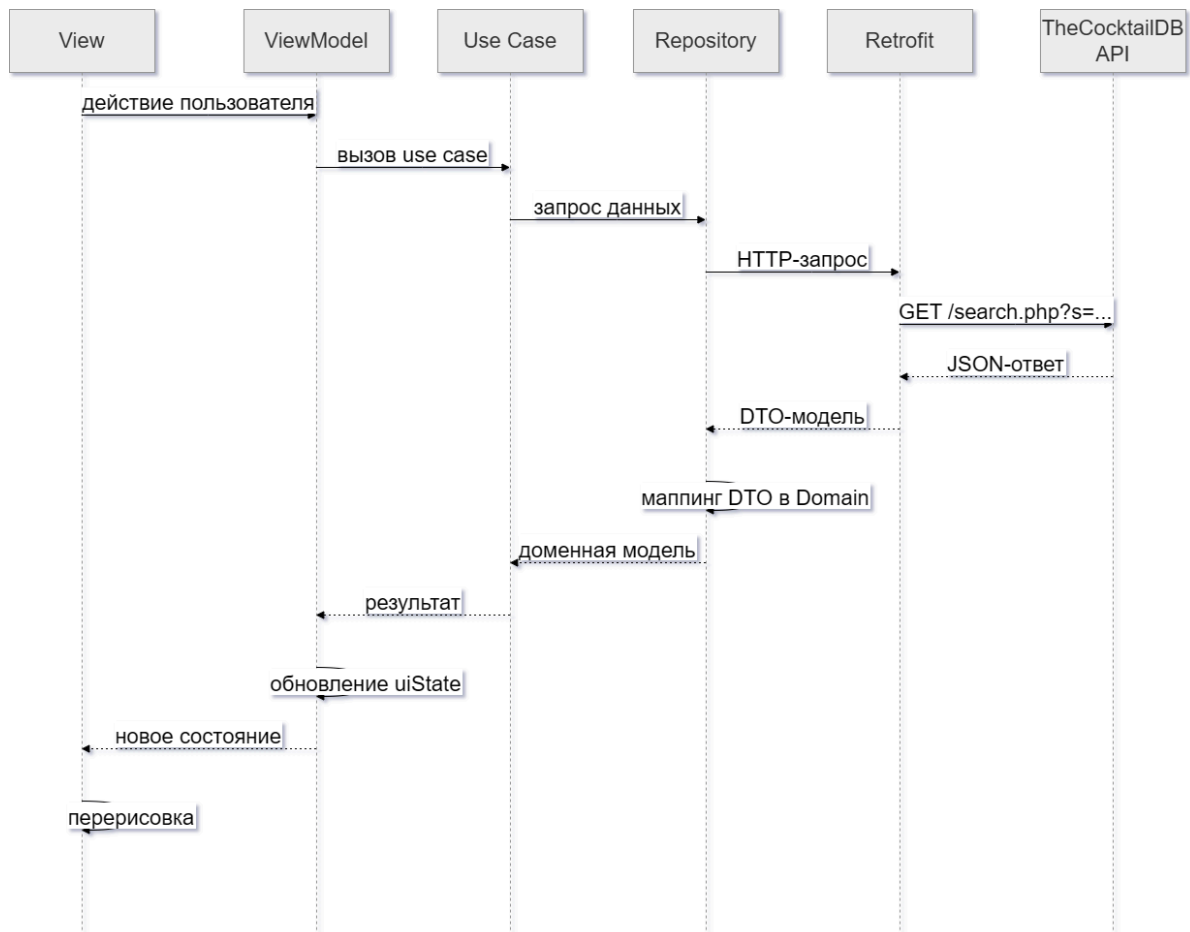


Рисунок 13 — Схема взаимодействия компонентов при выполнении сетевого запроса

Локальное хранение данных реализовано через Room и содержит три DAO-интерфейса: FavoriteDao, BarIngredientDao и RecentSearchDao. Все операции с базой данных выполняются в фоновом потоке и не блокируют основной поток интерфейса. Локальная база доступна всегда — вне зависимости от наличия интернет-соединения и состояния авторизации, что обеспечивает работу приложения в офлайн-режиме.

Синхронизация с Firebase Firestore активируется при авторизации пользователя через Google. При успешном входе выполняется объединение локальных и облачных данных с удалением дубликатов. Все последующие изменения

избранного и списка бара записываются одновременно в оба хранилища. При выходе из аккаунта приложение продолжает работу с локальными данными без потери функциональности.

3.3 Реализация модуля подбора рецептов по ингредиентам

Модуль подбора рецептов реализован в `MyBarViewModel`. При любом изменении списка ингредиентов бара `ViewModel` автоматически запускает алгоритм поиска подходящих коктейлей.

Для каждого ингредиента из списка бара через `FilterByIngredientUseCase` формируется отдельный запрос. Все запросы выполняются параллельно в рамках `supervisorScope` — это означает, что неудача одного запроса не прерывает выполнение остальных. После получения всех ответов результаты объединяются в единый список, из которого удаляются дубликаты по идентификатору коктейля.

Для предотвращения избыточных запросов при быстром добавлении нескольких ингредиентов подряд реализован механизм `debounce`: алгоритм запускается только спустя 400 мс после последнего изменения списка бара. Это достигается через оператор `debounce` в `Kotlin Flow`, на который подписан `ViewModel`.

На рисунке 14 представлена схема работы модуля при обновлении списка ингредиентов.

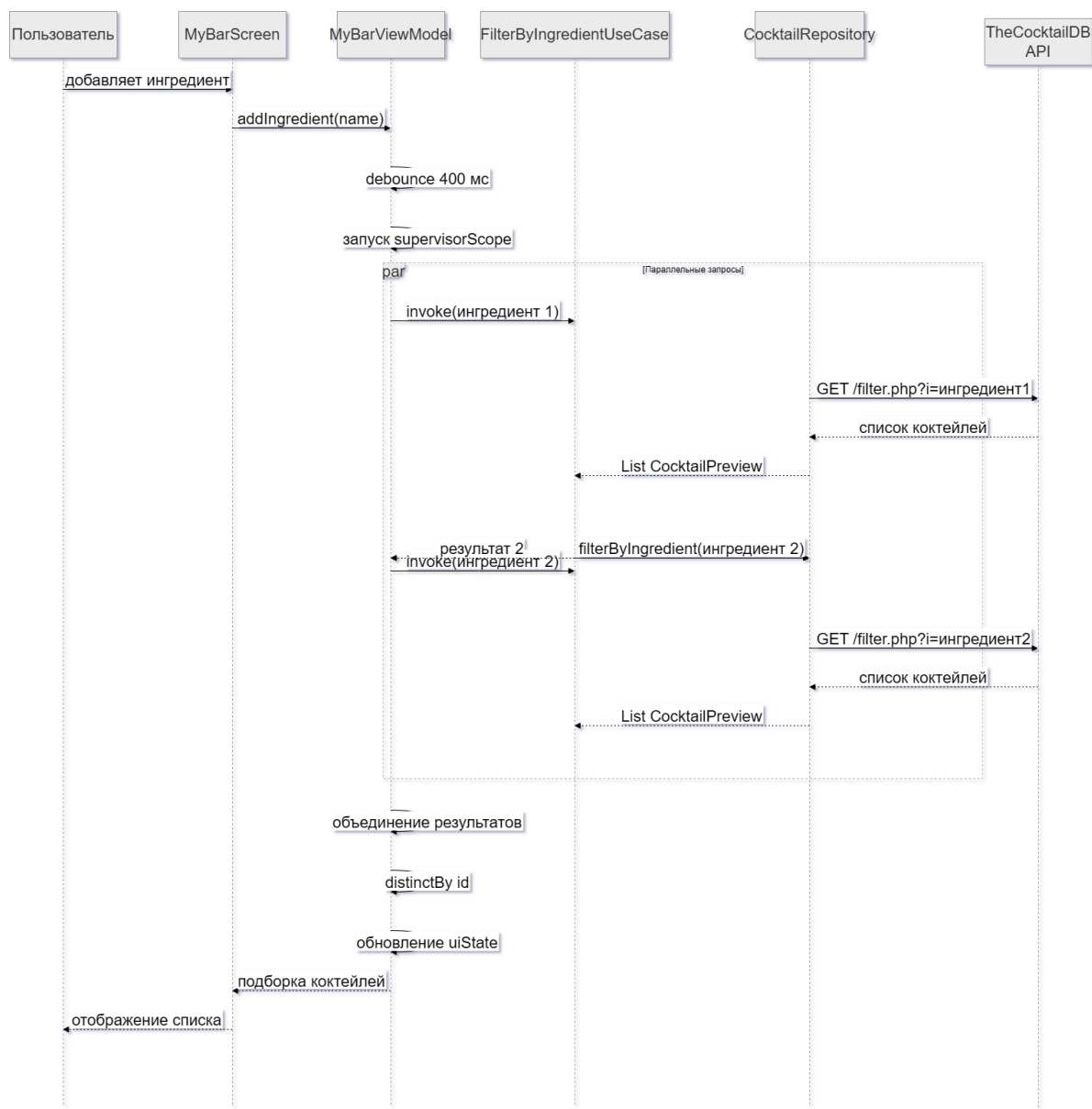


Рисунок 14 — Схема работы модуля при обновлении списка ингредиентов

3.4 Выводы по реализационной части

В результате выполнения реализационной части работы были получены следующие результаты:

- реализован пользовательский интерфейс с пятью основными экранами и детальной карточкой коктейля;
- реализована интеграция с внешним источником данных, обеспечивающая получение рецептов, категорий и ингредиентов;

- реализован модуль подбора рецептов по ингредиентам на основе параллельных запросов с объединением результатов и устранением дубликатов;
- реализовано локальное хранение пользовательских данных через Room и опциональная облачная синхронизация через Firebase Firestore;
- реализована авторизация через Google Sign-In с корректной обработкой состояний входа и выхода из аккаунта.

Все компоненты системы реализованы в соответствии со спроектированной архитектурой. Полученный результат соответствует функциональным и нефункциональным требованиям, сформулированным в проектной части.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано мобильное приложение для поиска рецептов коктейлей с функцией подбора по ингредиентам, имеющимся у пользователя дома.

В аналитической части был проведён анализ предметной области, рассмотрены существующие аналоги — Cocktail Flow, Drinkly и My Bar — и выявлены их основные недостатки. Установлено, что ни одно из рассмотренных решений не сочетает одновременно большую базу рецептов, бесплатный подбор по ингредиентам и синхронизацию данных без обязательной регистрации, что подтвердило актуальность разработки.

В проектной части были сформулированы функциональные и нефункциональные требования, обоснован технологический стек, спроектирована архитектура на основе Clean Architecture и MVVM, разработана структура локальной базы данных и определена схема облачного хранилища. Отдельное внимание было уделено проектированию модуля подбора рецептов по ингредиентам — наиболее технически нетривиальной части приложения, потребовавшей разработки клиентского алгоритма параллельного выполнения запросов с последующим объединением и дедупликацией результатов.

В реализационной части все спроектированные компоненты были воплощены в рабочее приложение. Реализованы пять основных экранов с корректными пустыми состояниями, интеграция с внешним источником данных, модуль параллельного подбора рецептов, локальное хранение данных через Room и облачная синхронизация через Firebase Firestore.

Основные функциональные требования, связанные с поиском, просмотром рецептов, избранным, баром пользователя и синхронизацией данных, были реализованы в полном объёме. Разработанное приложение решает реальную пользовательскую задачу: позволяет быстро найти подходящий рецепт коктейля исходя из того, что уже есть дома, без

необходимости просматривать сторонние сайты и вручную сопоставлять состав рецептов с содержимым домашнего бара.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. TheCocktailDB — бесплатная база данных рецептов коктейлей с API : [сайт]. – URL: <https://www.thecocktaildb.com/api.php> (дата обращения: 01.06.2026). – Текст : электронный.
2. TheCocktailDB — документация по фильтрации : [сайт]. – URL: <https://www.thecocktaildb.com/api.php> (дата обращения: 01.06.2026). – Текст : электронный.
3. Cocktail Flow — страница приложения в Google Play : [сайт]. – URL: <https://play.google.com/store/apps/details?id=com.robinhood.cocktailflow> (дата обращения: 01.06.2026). – Текст : электронный.
4. Drinkly — страница приложения в Google Play : [сайт]. – URL: <https://play.google.com/store/apps/details?id=com.drinkly> (дата обращения: 01.06.2026). – Текст : электронный.
5. My Bar — страница приложения в Google Play : [сайт]. – URL: <https://play.google.com/store/apps/details?id=com.mybar> (дата обращения: 01.06.2026). – Текст : электронный.
6. Android Developers — Kotlin-first approach : [сайт]. – URL: <https://developer.android.com/kotlin/first> (дата обращения: 01.06.2026). – Текст : электронный.
7. Android Developers — Jetpack Compose UI App Development Toolkit : [сайт]. – URL: <https://developer.android.com/compose> (дата обращения: 01.06.2026). – Текст : электронный.
8. Koin — The Kotlin Dependency Injection Framework : [сайт]. – URL: <https://insert-koin.io> (дата обращения: 01.06.2026). – Текст : электронный.
9. Android Developers — Save data in a local database using Room : [сайт]. – URL: <https://developer.android.com/training/data-storage/room> (дата обращения: 01.06.2026). – Текст : электронный.
10. Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. — Prentice Hall, 2017. — 432 с. — Текст : непосредственный.